

Object-Oriented Metrics

Instructor: Muhammad Zeeshan Nazar

Object-Oriented Metrics

- CK Metrics
 - Proposed by Chidamber and Kemerer
 - Class-based metrics
- LK Metrics
 - Proposed by Lorenz and Kidd
 - Class-based and operation-based
- MOOD Metrics
 - Proposed by Harrison, Counsell, and Nithi
 - Class-based

What are Object-Oriented Metrics?

- Object-Oriented metrics are measurements that help us evaluate:
 - Code quality
 - Maintainability
 - Complexity
 - Reusability
 - Testability
 - Overall software design health

They help detect **bad design, code smells, and violations of OO principles (SOLID, etc.)**.

CK Metrics

- CK metrics, also known as **Chidamber and Kemerer metrics**, are a suite of object-oriented metrics designed to assess the **quality of software design**, particularly for object-oriented systems.
- These metrics help in identifying **design flaws** and predicting the **maintainability**, **reliability**, and **reusability** of a system.

CK Metrics

- **Weighted methods per class (WMC)**

- WMC is the **sum of complexities of all methods** within a class.
- If every method has a complexity of 1, WMC is simply the count of methods in the class.
- They do not specify the specific complexity metric to use (e.g., cyclomatic complexity).
- High WMC indicates a **high level of functionality and complexity**, which may make a class harder to understand, test, and maintain.
- WMC should be kept **low**.
- Example: Suppose a Customer class in an e-commerce application has methods like addToCart, removeFromCart, checkout, and updateProfile. If each method is of equal complexity, and there are four methods, WMC = 4.
- Higher WMC $\Rightarrow$ more complex class, harder to maintain, violates SRP.

CK Metrics

- **Weighted methods per class (WMC)**

- WMC is a predictor of how much **time** and **effort** is required to develop and maintain the class. The higher the value, the more effort required.
- A large number of methods also means a greater potential impact on derived classes, since the derived classes inherit the methods of the base class.
- Search for high WMC values to spot classes that could be restructured into several smaller classes.

CK Metrics

- Depth of inheritance tree (DIT)
 - DIT is the length of the longest path from a given class to the root class in the inheritance hierarchy.
 - As DIT increases, the lower classes in the hierarchy inherit a greater number of data and methods, thus making their behavior more difficult to understand and causing testing to require more effort.
 - A large DIT value implies greater design complexity, but also greater reuse.
 - Example: Consider a class hierarchy where Animal is the root class, Mammal inherits from Animal, Dog inherits from Mammal, and Bulldog inherits from Dog. Here, Bulldog has a DIT of 3.
 - Higher DIT $\Rightarrow$ more inherited functionality, more complexity.

```
class A {};           // DIT = 0
class B : public A {}; // DIT = 1
class C : public B {}; // DIT = 2
```

CK Metrics

- **Number of children (NOC)**

- NOC is the **number of immediate subclasses** inheriting from a particular class.
- **Higher NOC** values suggest more **complexity**, as the class must support more inheriting classes.
- High NOC can be a sign of **over-generalization** or it may indicate that a class is designed for extensibility.
- Example: Suppose we have a Shape class, and three classes inherit from it: Circle, Square, and Triangle. Here, NOC for Shape is 3.
- Too high $\Rightarrow$ risk of misuse, harder to change

```
class Shape {};
```

```
class Circle : public Shape {};
```

```
class Square : public Shape {};
```

```
class Triangle : public Shape {};
```


CK Metrics

- **Coupling between object classes (CBO)**

- CBO measures the **number of other classes to which a given class is coupled**, either by using their methods or properties.
- **High coupling** makes the code **less modular** and **harder to maintain or test**, as changes in one class may impact other classes.
- Example: If a Customer class in a banking application calls methods from Account, Loan, and Transaction classes, CBO for Customer would be 3.
- Higher coupling $\Rightarrow$ harder to maintain, violates DIP & OCP.

```
class Engine {};  
class Wheels {};
```

```
class Car {  
    Engine e;    // depends on Engine  $\rightarrow$  1  
    Wheels w;    // depends on Wheels  $\rightarrow$  2  
};
```

CK Metrics

- **Response for a class (RFC)**

- The number of methods that can potentially be executed in response to a message received by an object of a given class.

$$RFC = M + R$$

- M = number of methods in the class
- R = number of remote methods directly called by methods of the class
- As the RFC value increases, testing effort and design complexity also increase.
- RFC should be kept low.

```
class Math {  
public:  
    int add(int x, int y) { return x + y; }  
    int square(int x) { return x * x; }  
};
```

```
class Demo {  
public:  
    void process() {  
        Math m;  
        m.add(1,2);    // calling add()  
        m.square(3);   // calling square()  
    }  
};
```

CK Metrics

- **Lack of cohesion in methods (LCOM)**

- Lack of Cohesion in Methods (LCOM) measures **how closely related the methods in a class are**, based on the instance variables (attributes) they access.
- One common approach to calculate LCOM is based on **method pairs**, which calculates cohesion by examining how frequently methods share attributes.

CK Metrics

- **Lack of cohesion in methods (LCOM)**

- Identify pairs of methods.
- For each pair:
 - Count it as "connected" if they access at least one common attribute.
 - Count it as "disconnected" if they do not share any attributes.
- Calculate LCOM as follows:

LCOM = Number of disconnected method pairs – Number of connected method pairs

- If LCOM is positive, it indicates a lack of cohesion (i.e., methods are disconnected).
- If LCOM is zero or negative, the class has good cohesion.

CK Metrics

```
class Book {  
public:  
    string title;  
    string author;  
    int pages;  
  
    void printTitle() { cout << title; }  
    void printAuthor() { cout << author; }  
    void printPages() { cout << pages; }  
    void updatePages(int p) { pages = p; }  
    void printSummary() { cout << title << " by " << author; }  
};
```

CK Metrics

Method	Attributes used
printTitle()	title
printAuthor()	author
printPages()	pages
updatePages()	pages
printSummary()	title, author

CK Metrics

Pair	Shared Attribute?	Count
printTitle, printAuthor	No	P
printTitle, printPages	No	P
printTitle, updatePages	No	P
printTitle, printSummary	Yes (title)	Q
printAuthor, printPages	No	P
printAuthor, updatePages	No	P
printAuthor, printSummary	Yes (author)	Q
printPages, updatePages	Yes (pages)	Q
printPages, printSummary	No	P
updatePages, printSummary	No	P

LK Metrics

- **LK metrics**, developed by **Lorenz and Kidd**, are another popular suite of object-oriented design metrics.
- Unlike CK metrics, which focus on assessing object-oriented design quality in general, LK metrics are often more specific to **code characteristics**, particularly in **class structure and object relationships**.
- These metrics help provide insights into design cohesion, modularity, and complexity.

LK Metrics

- **Class size (CS)**

- The total number of **methods** (both inherited and local) plus the total number of **attributes** (both inherited and local) encapsulated by a given class.
- Inherited members should be weighted more heavily than local members.
- Large values of CS could indicate that the class is too large, that it encapsulates too much behavior, structure, and responsibility.
- High CS values may also indicate lower reusability.

LK Metrics

- **Number of operations overridden by a subclass (NOO)**
 - A count of the methods in subclasses that have been redefined.
 - Large NOO values could indicate a design problem, since the model of the class seems to be violated.
 - Since a subclass should be a specialization of its super classes, it should primarily extend the services [operations] of the super classes. This should result in unique new method names.

LK Metrics

- **Number of operations added by a subclass (NOA)**
 - A count of the new methods appearing in subclasses.
 - A large NOA value could indicate a design abstraction violation.
 - As CK DIT increases, NOA should decrease.

LK Metrics

- **Specialization index (SI)**

- The degree to which subclasses are differentiated from super classes.
- SI is computed as NOO multiplied by the level at which the class resides in the inheritance hierarchy divided by the total number of methods defined by the class.

$$\text{SI} = (\text{NOO} * \text{depth}) / (\text{total number of class methods})$$

- A high SI could indicate a lack of conformance to superclass abstractions.

LK Metrics

- **Average operation size (OS_{avg})**

- Average Operation Size (OS_{avg}) is a metric used to measure the **size** or **complexity** of operations (methods/functions) within a class.
- It quantifies the average number of **messages sent** by methods within a specific class.
- When a method within a class sends out a high number of messages, it means that the method is interacting with various other methods, classes, or components.
- A high OS_{avg} value suggests that the methods within a class are doing a lot of work by interacting with many other parts of the system.
- A high OS_{avg} value can indicate a poor allocation of responsibility within the class.

LK Metrics

- **Operation complexity (OC)**

- Operation Complexity (OC) is a metric used to evaluate the **complexity of methods or operations** within a class.
- It often measures aspects like cyclomatic complexity, which assesses the number of independent paths within a method.
- OC should be kept as low as possible.

LK Metrics

- **Average number of parameters per operation (NP_{avg})**
 - The Average number of parameters per operation (NP_{avg}) is a metric that measures the average size of parameter lists in methods or functions within a class or system.
 - Should be kept low

Reference

- Software Engineering: Theory and Practice, Shari Lawrence Pfleeger and Joanne M. Atlee, 4th Edition, Pearson, 2009.